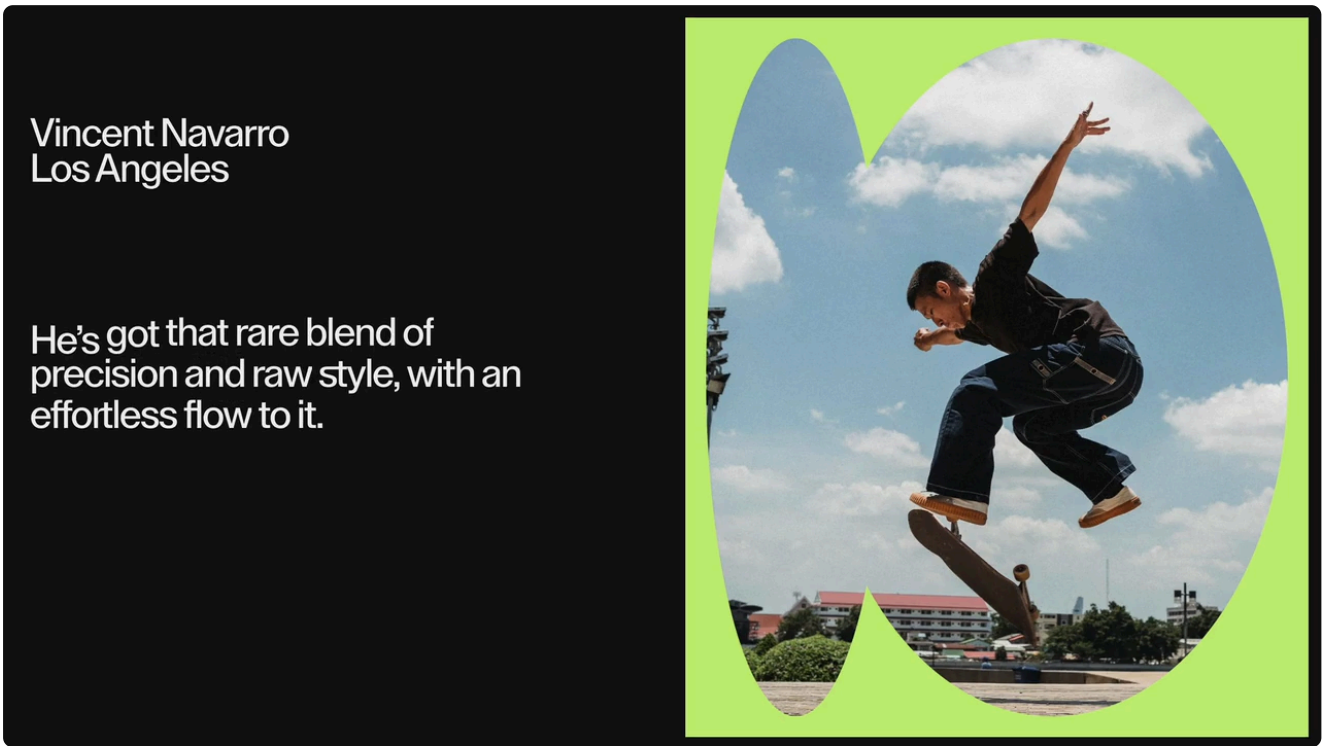
Made With Gsap

006
Progressive sentences



In this effect, we will make a paragraph disappear word by word while making the next one appear in the same way. Our logic will be dynamic: if we add one or more paragraphs, the effect will continue to work. Thank you, JavaScript!

HTML Structure

The structure here is relatively simple: the `container` div contains the different paragraphs we are going to animate.

```
<div class="pin-height">
    <div class="container">
        <p class="paragraph">“We met Vincent at Venice Beach’s Skate Park.</p>
        <p class="paragraph">He’s got that rare blend of precision.</p>
        <p class="paragraph">He’s been at it since he was a kid.”</p>
    </div>
</div>
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

height over which our `container` remains pinned while scrolling. I chose a value of `400vh` to ensure the effect plays out fully. If I reduce this value, the paragraphs will appear faster since there will be less distance to scroll.

```
.mwg_effect006 .pin-height{
    height: 400vh;
}
.mwg_effect006 .container{
    height: 100vh;
}
```

To make sure all paragraphs are positioned in the same place within our component, I set `position: absolute` on all paragraphs except the first one. The first paragraph takes up real space in case other elements follow in the page's flow.

```
.mwg_effect006 .paragraph:not(:first-child) {
    position: absolute;
    top: 0;
}
```

By default, we hide the words of each paragraph except for the first one, so that only the first paragraph is visible when the effect starts. The words will be wrapped inside a `.word` mask and then inside a `span` for animation.

```
.mwg_effect006 .paragraph:not(:first-child) .word span {
    transform: translate(0, -100%);
}
```

Splitting paragraphs into words

In JavaScript, I start by applying a utility function to each paragraph. This function returns a sentence with each word wrapped inside `span` tags.

```
const root = document.querySelector('.mwg_effect006')
const paragraphs = root.querySelectorAll('.paragraph')
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Now let's take a look at the `wrapWordsInSpan()` function. I use the `split()` method to begin with: this method divides the original text into an array, using spaces `(' ')` as separators. The map() method then wraps each word with . Finally, I reconstruct the sentence by joining the words back together with spaces. This new string (now containing its DOM structure) is then applied back to the target element.

```
function wrapWordsInSpan(element) {
    const text = element.textContent;
    element.innerHTML = text
        .split(' ')
        .map(word => `<span class="word"><span>${word}</span></span>`)
        .join(' ');
}
```

Each word is wrapped in a `span`, which is itself inside another `span`. The parent one acts as a mask, while the child one holds the hidden word.

Pinning the container

Next, I pinned the `container` div over the full height of `pin-height` using a `ScrollTrigger.create()` instance. This instance allows us to define when the container should stay pinned.

```
const pinHeight = root.querySelector('.pin-height')
const container = root.querySelector('.container')

ScrollTrigger.create({
    trigger: pinHeight, // Listening to pinHeight
    start: 'top top',
    end: 'bottom bottom',
    pin: container, // The pinned section
    scrub: true // Syncs with scrolling
})
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

hes the top of the viewport, we

Made With Gsap

ght scrolls past the bottom of the viewport. This ensures the `container` remains fully visible over this distance.

Animating the Paragraphs

For the words animation of each paragraph, I declare a `gsap.timeline()`. This timeline will always play throughout the height of the `pinHeight` div.

```
const tl = gsap.timeline({
    scrollTrigger: {
        trigger: pinHeight,
        start: 'top top',
        end: 'bottom bottom',
        scrub: true // progresses with the scroll
    }
})
```

All tweens within the timeline will progress using the same `scrollTrigger` parameters. Since every tween in the timeline has the same `duration`, each step will be evenly distributed across the full animation length.

Now, we need to make the words of one paragraph disappear while the words of the next paragraph appear at the same time. We repeat this process for each paragraph using the native `forEach()` function.

```
paragraphs.forEach((paragraph, index) => {
    tl.to(paragraphs[index].querySelectorAll('.word span'), {
        y: '100%', // Hiding words of paragraphs[index]
        stagger: 0.2,
        duration: 1,
        ease: 'power4.in',
    })
    tl.to(paragraphs[index+1].querySelectorAll('.word span'), {
        y: '0%', // Revealing words of paragraphs[index+1]
        duration: 1,
        delay: 1.4,
        stagger: 0.2,
        ease: 'power4.out',
    }, '<') // Means that the animation starts at the same time as the previous tween
})
```

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS
Made With Gsap

; that the default `duration` of element animates for 0.5 seconds, and I stagger their start times by 0.5 seconds, that means they'd be perfectly sequenced. When we set `scrub: true` on a ScrollTriggered animation, it just squishes/stretches it to fit perfectly between the start and end scroll positions.

So if we set `stagger: 1` here, given that `duration` is `1`, each word would appear exactly when the previous one has finished moving. Reducing this value allows multiple words to overlap in motion.

Right now, the last paragraph fully disappears at the end of the animation. Since the effect starts with the first paragraph already visible, we should end it by keeping the last paragraph on screen visible. To achieve this, we just have to add a condition to check if there's a next paragraph after the current one:

```
paragraphs.forEach((paragraph, index) => {
    if(paragraphs[index+1]) { // Is there a next paragraph?
        tl.to(paragraphs[index].querySelectorAll('.word span'), {
            y: '100%', // Hiding words of paragraph[index]
            stagger: 0.2,
            duration: 1,
            ease: 'power4.in',
        }, `step${index}`) // Both tweens will play at the same time
        tl.to(paragraphs[index+1].querySelectorAll('.word span'), {
            y: '0%', // Revealing words of paragraph[index+1]
            duration: 1,
            delay: 1.4,
            stagger: 0.2,
            ease: 'power4.out',
        }, `step${index}`) // Both tweens will play at the same time
    }
})
```

Go further

For this kind of effect with overlapping elements, it's important to use the same easing family. Otherwise, words from the next paragraph might appear on top of words that are still fading out. Also, keep in mind that using paragraphs of similar length makes the effect more consistent.

In this example, I used the Lenis smooth scroll to further enhance the fluidity and sliding effect. If you'd like to implement it as well, visit our Go Further page to learn how to easily add it.

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

HTML CSS JAVASCRIPT

```html
<section class="mwg_effect006">
    <div class="pin-height">
        <div class="container">
            <p class="name">Vincent Navarro <br />Los Angeles</p>
            <div class="left">
                <p class="paragraph">“We met Vincent at Venice Beach’s Skate Park, and
                <p class="paragraph">He’s got that rare blend of precision and raw styl
                <p class="paragraph">He’s been at it since he was a kid and he might tu
            </div>

            <img class="right" src="assets/medias/photo.jpg" alt="">
        </div>
    </div>
</section>
```

Wanna learn something else?

BACK TO THE COLLECTION

BATCH 001 — 050 EFFECTS

COPY/PASTE TO WEBFLOW AVAILABLE SEE HOW EASY IT IS

Made With Gsap

Your email

SUBMIT

By submitting your email, you'll be the first to know about upcoming updates for Made With Gsap. You can unsubscribe at any time.

SOCIAL

X(TWITTER)
INSTAGRAM
BLUESKY
LINKEDIN

PAGES

HOME
COLLECTION
FAQ
PRICING

CONTACT

REACH US
SUPPORT

Made With Gsap

2025©

PERMISSIONS & TERMS

PRIVACY POLICY